

Reporte Académico Final: Proyecto Integrador (Fase II)

Curso: Solución de problemas con programación computacional

Actividad: Entrega Final del Proyecto (Fase II)

Establecimiento: Taquería Las Brasas de Oaxaca

1. Propósito de Creación y Análisis Organizacional

Propósito del Sistema

El presente software fue desarrollado con el propósito de automatizar las operaciones comerciales del negocio familiar **Taquería Las Brasas de Oaxaca**, sustituyendo el método tradicional de registro manual en notas de papel (comandas) y el cálculo aritmético mediante calculadora al finalizar la jornada.

Problemática Operativa Atendida

Tradicionalmente, el establecimiento operaba mediante comandas físicas, lo que generaba las siguientes deficiencias operativas:

- **Errores frecuentes** en el cobro y cálculo de subtotales durante las horas pico de atención.
- **Pérdida o daño** de tickets de papel, impidiendo un control histórico confiable de las ventas.
- **Lentitud en la atención** al cliente debido a la dependencia de procesos manuales de registro.
- **Ausencia de seguridad** con mecanismos de autenticación de usuarios y respaldo persistente de la información financiera.

Para resolver esto, la aplicación implementa un sistema en consola robusto que gestiona el acceso de usuarios, procesa pedidos mediante estructuras matriciales, aplica reglas de negocio específicas, valida entradas mediante excepciones y garantiza la persistencia automática de los datos en archivos externos.

2. Definición del Problema y Reglas de Negocio

- **Autenticación y Registro:** El acceso al sistema se realiza mediante un nombre de usuario y una contraseña, los cuales se almacenan y validan de forma persistente en archivos de texto independientes (usuarios.txt y contras.txt).
- **Menú y Promociones:** El sistema ofrece productos base de la taquería (tacos, quesadillas, alambres, aguas frescas y refrescos).
- **Ajuste por Pieza Individual (cargoeextra):** Si un cliente solicita una única unidad de un

producto cuyo precio de menú está configurado para un paquete de varias piezas (por ejemplo, tacos o quesadillas), el sistema aplica de manera automática un cargo extra de **+\$10.00 MXN** al precio base unitario.

- **Persistencia Diaria:** Cada corte de caja y ticket de mesa se almacena en un archivo de texto fechado dinámicamente (ventas_dia_mes_anio.txt) e indexado en un registro general (registro_ventas.txt).

3. Cumplimiento de Requerimientos Técnicos

A continuación se detalla cómo se implementaron los requerimientos obligatorios establecidos en la guía de la actividad (omitiendo la depuración con PDB por tratarse de un apartado opcional):

1. **Identificación de Usuario:** El sistema solicita el nombre o apodo del operador al iniciar o mediante el módulo de gestión de cuentas.
2. **Bienvenida Dinámica:** Se generan saludos formales personalizados incorporando el nombre del usuario mediante operadores de cadenas.
3. **Pantalla de Carga:** Se implementó la función dedicada pantalla_carga(), la cual genera una pausa interactiva de 5 segundos con un aviso dinámico en consola antes de desplegar el menú principal.
4. **Menú como Matriz:** El menú principal se encuentra estructurado mediante una matriz lógica en Python (menu_matriz) y es controlado de forma continua mediante un ciclo while.
5. **Control de Inactividad del Usuario:** Se utiliza la función verificar_inactividad_con_for() apoyada en un ciclo for para evaluar si han transcurrido más de 10 minutos sin interacción. Ante la inactividad, solicita confirmación escribiendo explícitamente "sí" o "no".
6. **Captura de Fecha Estructurada:** La función capturar_fecha() solicita el día, mes y año, almacenándolos estrictamente en una tupla bajo la sintaxis Fecha = (día, mes, año), la cual se integra automáticamente en la denominación y contenido de los archivos de ventas.
7. **Persistencia en Archivos de Texto:** El sistema lee, escribe y anexa información de forma permanente utilizando múltiples archivos externos (usuarios.txt, contras.txt, registro_ventas.txt y archivos diarios de ventas), permitiendo consultar su contenido mediante índices numéricos.
8. **Control de Excepciones del Sistema:** Se integran bloques try-except para capturar errores de ejecución como FileNotFoundError (archivos inexistentes) y ValueError (ingreso de datos alfanuméricos en campos numéricos).
9. **Comentarios de Calidad:** El código fuente cuenta con comentarios extensos y descriptivos que facilitan su comprensión y mantenimiento.

4. Código en Python

```
import time
```

```
# Define la constante global para el tiempo máximo de inactividad (10 minutos)
```

```
TIEMPO_MAXIMO_INACTIVIDAD = 10 * 60 # 600 segundos
```

```
def verificar_inactividad_con_for(ultima_actividad):
    # Si no han pasado 10 min sin actividad, continúa la sesión
    if time.time() - ultima_actividad < TIEMPO_MAXIMO_INACTIVIDAD:
        return ultima_actividad

    print("\nAlerta: Se han detectado 10 minutos de inactividad.")
    for intento in range(1, 4):
        respuesta = (
            input('¿Deseas continuar en la sesión? Escribe "sí" o "no": ')
            .strip()
            .lower()
        )
        if respuesta == "sí":
            print("Reanudando sesión...")
            return time.time()
        if respuesta == "no":
            print("Regresando al inicio de sesión...")
            return "reiniciar"
        print(
            f'Intento {intento}/3. Respuesta no válida. Debe escribir "sí" o "no".'
        )

    print("Demasiados intentos fallidos. Cerrando sesión.")
    return "reiniciar"
```

```
def pantalla_carga():
    # Muestra una animación simple de carga al iniciar el programa.
    print("\n[SISTEMA] Iniciando programa...")
    for segundo in range(1, 6):
        print(f"Cargando... {segundo}/5 segundos")
        time.sleep(1)
    print("Sistema cargado con éxito\n")
```

```
def gestionar_usuarios():
    # Gestiona el registro e inicio de sesión guardando datos en archivos separados
    archivo_usuarios = "usuarios.txt"
    archivo_contras = "contras.txt"

    try:
        with open(archivo_usuarios, "r") as f_u, open(archivo_contras, "r") as f_c:
            usuarios_registrados = f_u.read().splitlines()
```

```

        usuarios_contras = f_c.read().splitlines()
except FileNotFoundError:
    with open(archivo_usuarios, "w") as f_u, open(archivo_contras, "w") as f_c:
        usuarios_registrados = []
        usuarios_contras = []

print("=" * 40)
print("  ACCESO DE USUARIOS  ")
print("=" * 40)
print("1. Iniciar sesion")
print("2. Registrar nueva cuenta")

opcion = input("Seleccione una opción (1 o 2): ")

if opcion == "1":
    usuario = input("Ingrese su nombre de usuario: ")
    contra = input("Ingrese su contraseña de usuario: ")

    if usuario in usuarios_registrados:
        indice = usuarios_registrados.index(usuario)
        if usuarios_contras[indice] == contra:
            print(f"\nBienvenido de nuevo, {usuario}.")
            return usuario

    print(
        "\nEl usuario no esta registrado o la contraseña es incorrecta. Intente de nuevo o
registre."
    )
    return gestionar_usuarios()

elif opcion == "2":
    nuevo_usuario = input("Elija un nombre de usuario nuevo: ")
    nueva_contra = input("Elija una contraseña de usuario nueva: ")

    if nuevo_usuario in usuarios_registrados:
        print("\nEse usuario ya existe. Intente iniciar sesión.")
        return gestionar_usuarios()
    else:
        with open(archivo_usuarios, "a") as f_u, open(
            archivo_contras, "a"
        ) as f_c:
            f_u.write(nuevo_usuario + "\n")
            f_c.write(nueva_contra + "\n")

```

```

    print(
        f"\nCuenta creada y guardada con exito, bienvenido, {nuevo_usuario}."
    )
    return nuevo_usuario

else:
    print("Opción invalida.")
    return gestionar_usuarios()

def capturar_fecha():
    # Captura y devuelve una tupla con la fecha de operación ingresada.
    print("\n--- CAPTURA DE FECHA DE OPERACIÓN ---")
    dia = input("Ingrese el día (ej. 12): ")
    mes = input("Ingrese el mes (ej. 06): ")
    anio = input("Ingrese el año (ej. 2026): ")

    Fecha = (dia, mes, anio)
    print(f"Fecha registrada: {Fecha[0]}/{Fecha[1]}/{Fecha[2]}")
    return Fecha

def registrar_en_indice(nombre_archivo):
    # Registra el nombre del archivo en 'registro_ventas.txt' con un número único.
    archivo_indice = "registro_ventas.txt"
    archivos_existentes = []

    try:
        with open(archivo_indice, "r") as f:
            archivos_existentes = f.read().splitlines()
    except FileNotFoundError:
        pass

    if nombre_archivo not in archivos_existentes:
        archivos_existentes.append(nombre_archivo)
        with open(archivo_indice, "w") as f:
            for i, nombre in enumerate(archivos_existentes, start=1):
                f.write(f"{i}-{nombre}\n")

def taqueria_pedidos(fecha_actual):
    # Controla el registro de pedidos por mesa y calcula el total de ventas.
    total_dia = 0
    num_pedidos = 0
    continuar_cliente = "si"

```

```

while continuar_cliente.lower() == "si":
    print("=====")
    print("  TAQUERÍA LAS BRASAS DE OAXACA  ")
    print("=====")

    try:
        numero_mesa = int(input("Ingrese el número de mesa: "))
    except ValueError:
        print("Número de mesa inválido.")
        continue

    subtotal = 0
    opcion = 0

    while opcion != 6:
        print("\n--- MENÚ DE PLATILLOS ---")
        print("1. Orden Tacos al Pastor (3 pzs) - $60")
        print("2. Orden Quesadillas Oaxaqueñas (2 pzs) - $50")
        print("3. Alambre Especial de la Casa - $110")
        print("4. Agua Fresca (1 L) - $30")
        print("5. Refresco (600 ml) - $25")
        print("6. Finalizar pedido de la mesa")

        try:
            opcion = int(input("Seleccione una opción (1-6): "))
        except ValueError:
            print("Opción inválida.")
            continue

    if 1 <= opcion <= 5:
        try:
            cantidad = int(input("Ingrese la cantidad solicitada: "))
        except ValueError:
            print("Cantidad inválida.")
            continue

        if cantidad > 0:
            cargoextra = 0
            precio = 0

            if opcion == 1:
                precio = 60
                if cantidad == 1:

```

```

        resp = input(
            "¿Desea 1 pieza individual? (s/n): "
        ).lower()
        if resp == "s":
            precio = 25
            cargoextra = 10
    elif opcion == 2:
        precio = 50
        if cantidad == 1:
            resp = input(
                "¿Desea 1 pieza individual? (s/n): "
            ).lower()
            if resp == "s":
                precio = 35
                cargoextra = 10
    elif opcion == 3:
        precio = 110
    elif opcion == 4:
        precio = 30
    elif opcion == 5:
        precio = 25

    subtotal += precio * cantidad
    if cargoextra > 0:
        subtotal += cargoextra
        print("-> Se aplicó un cargo extra de $10.00 MXN.")
        print(f"-> Subtotal mesa: ${subtotal}")
    else:
        print("La cantidad debe ser mayor a 0.")
    elif opcion == 6:
        break
    else:
        print("Opción inválida.")

if subtotal > 0:
    total_pedido = subtotal
    total_dia += total_pedido
    num_pedidos += 1

print(f"\nRESUMEN MESA {numero_mesa} | Total: ${total_pedido}")
nombre_archivo = f"ventas_{fecha_actual[0]}_{fecha_actual[1]}_{fecha_actual[2]}.txt"

with open(nombre_archivo, "a") as archivo:

```

```

        archivo.write(
            f"Fecha: {fecha_actual[0]}/{fecha_actual[1]}/{fecha_actual[2]} | Mesa: {numero_mesa}
| Total: ${total_pedido}\n"
        )

```

```

        registrar_en_indice(nombre_archivo)
        print(f"-> Venta guardada en '{nombre_archivo}'.")
    else:
        print("Pedido cancelado o sin ítems.")

```

```

continuar_cliente = input("¿Desea atender a otra mesa? (si/no): ")

```

```

print(
    f"\n--- CORTE DEL DÍA ---\nMesas atendidas: {num_pedidos} | Ventas totales: ${total_dia}"
)

```

```

def leer_archivo_texto():

```

```

    # Lee y muestra en consola el contenido de un archivo de ventas.
    print("\n--- LECTURA DE ARCHIVOS DE VENTAS ---")
    archivo_indice = "registro_ventas.txt"

```

```

    try:
        with open(archivo_indice, "r") as f:
            lineas = f.read().splitlines()
    except FileNotFoundError:
        print("No hay registros de ventas guardados todavía.")
        return

```

```

    if not lineas:
        print("El archivo de registro está vacío.")
        return

```

```

    print("\nDías de ventas registrados:")
    for linea in lineas:
        print(linea)

```

```

    try:
        num_seleccionado = int(
            input(
                "\nIngrese el número asignado al día de venta que desea ver: "
            )
        )
    except ValueError:

```

```
print("Debe ingresar un número válido.")
return
```

```
archivo_a_buscar = None
for linea in lineas:
    partes = linea.split("-", 1)
    if len(partes) == 2 and partes[0].strip() == str(num_seleccionado):
        archivo_a_buscar = partes[1].strip()
        break
```

```
if archivo_a_buscar:
    try:
        with open(archivo_a_buscar, "r") as archivo:
            print(f"\n--- Contenido de {archivo_a_buscar} ---")
            print(archivo.read())
            print("-----")
    except FileNotFoundError:
        print(
            f"El archivo '{archivo_a_buscar}' no se encuentra en el sistema."
        )
    else:
        print("Número asignado no válido.")
```

```
# Ejecución inicial de usuario y entorno
usuario = gestionar_usuarios()
print(f"\nHola, {usuario}")
pantalla_carga()
fecha_actual = capturar_fecha()
```

```
menu_matriz = [
    [1, "Iniciar Módulo de Pedidos"],
    [2, "Leer Archivo de Ventas (.txt)"],
    [3, "Salir"],
]
```

```
activo = True
ultima_actividad = time.time()
```

```
while activo:
    res = verificar_inactividad_con_for(ultima_actividad)
    if res == "reiniciar":
        usuario = gestionar_usuarios()
        ultima_actividad = time.time()
```

```

        continue
    else:
        ultima_actividad = res

    print("=" * 40)
    print("    MENÚ PRINCIPAL    ")
    print("=" * 40)
    for fila in menu_matriz:
        print(f"[{fila[0]]} {fila[1]}")
    print("=" * 40)

    try:
        opcion = int(input("Seleccione una opción: "))
        ultima_actividad = time.time()
    except ValueError:
        print("Ingrese un número válido.")
        ultima_actividad = time.time()
        continue

    if opcion == 1:
        taqueria_pedidos(fecha_actual)
        ultima_actividad = time.time()
    elif opcion == 2:
        leer_archivo_texto()
        ultima_actividad = time.time()
    elif opcion == 3:
        print(f"Hasta luego, {usuario}")
        activo = False
    else:
        print("Opción fuera de rango.")

```

5. Diseño Algorítmico (Pseudocódigo adaptado)

Proceso LasBrasasDeOaxaca

Definir opcion, activo Como Entero;

Definir usuarioActual Como Cadena;

Definir usuarios Como Cadena;

Definir contras Como Cadena;

Definir totalUsuarios Como Entero;

Definir fechaStr Como Cadena;

Dimension usuarios[100];

Dimension contras[100];

totalUsuarios <- 0;

// Cuenta por defecto para pruebas rápidas (opcional)

usuarios[0] <- "admin";

contras[0] <- "1234";

totalUsuarios <- 1;

// 1. Gestión de usuarios inicial

usuarioActual <- GestionarUsuarios(usuarios, contras, totalUsuarios);

Escribir "";

Escribir "Hola, ", usuarioActual;

// 2. Pantalla de carga y fecha

PantallaCarga();

fechaStr <- CapturarFecha();

activo <- 1;

Mientras activo == 1 Hacer

Escribir "";

Escribir "=====

Escribir " MENÚ PRINCIPAL ";

Escribir "=====

Escribir "[1] Iniciar Módulo de Pedidos";

Escribir "[2] Ver Resumen de Ventas del Día";

Escribir "[3] Salir";

Escribir "=====

Escribir Sin Saltar "Seleccione una opción: ";

Leer opcion;

Segun opcion Hacer

1:

 TaqueriaPedidos(fechaStr);

2:

 Escribir "";

 Escribir "--- REGISTRO DE VENTAS ---";

 Escribir "Fecha actual de operación: ", fechaStr;

 Escribir "Módulo de consulta activo en memoria.";

 Escribir "-----";

3:

 Escribir "Hasta luego, ", usuarioActual;

 activo <- 0;

De Otro Modo:

Escribir "Opción fuera de rango.";

FinSegun

FinMientras

FinProceso

SubProceso PantallaCarga()

Definir i Como Entero;

Escribir "";

Escribir "[SISTEMA] Iniciando programa...";

Para i <- 1 Hasta 5 Con Paso 1 Hacer

Escribir "Cargando... ", i, "/5 segundos";

Esperar 1 Segundos;

FinPara

Escribir "Sistema cargado con éxito";

Escribir "";

FinSubProceso

SubProceso fecha <- CapturarFecha()

Definir día, mes, año, fecha Como Cadena;

Escribir "";

Escribir "--- CAPTURA DE FECHA DE OPERACIÓN ---";

Escribir Sin Saltar "Ingrese el día (ej. 12): ";

Leer dia;

Escribir Sin Saltar "Ingrese el mes (ej. 06): ";

Leer mes;

Escribir Sin Saltar "Ingrese el año (ej. 2026): ";

Leer anio;

fecha <- dia + "/" + mes + "/" + anio;

Escribir "Fecha registrada: ", fecha;

FinSubProceso

SubProceso usuarioRetorno <- GestionarUsuarios(usuarios, contras, totalUsuarios Por Referencia)

Definir opcion Como Entero;

Definir user, pass Como Cadena;

Definir encontrado Como Logico;

Definir i Como Entero;

Definir usuarioRetorno Como Cadena;

Repetir

Escribir "=====";

Escribir " ACCESO DE USUARIOS ";

Escribir "=====";

Escribir "1. Iniciar sesion";

Escribir "2. Registrar nueva cuenta";

Escribir Sin Saltar "Seleccione una opción (1 o 2): ";

Leer opcion;

Si opcion == 1 Entonces

Escribir Sin Saltar "Ingrese su nombre de usuario: ";

Leer user;

Escribir Sin Saltar "Ingrese su contraseña de usuario: ";

Leer pass;

encontrado <- Falso;

Para i <- 0 Hasta totalUsuarios - 1 Con Paso 1 Hacer

Si usuarios[i] == user Y contras[i] == pass Entonces

encontrado <- Verdadero;

FinSi

FinPara

Si encontrado Entonces

Escribir "";

Escribir "Bienvenido de nuevo, ", user;

usuarioRetorno <- user;

opcion <- 3; // Sale del repetir

Sino

Escribir "";

Escribir "El usuario no esta registrado o la contraseña es incorrecta. Intente de nuevo.";

FinSi

Sino

Si opcion == 2 Entonces

Escribir Sin Saltar "Elija un nombre de usuario nuevo: ";

Leer user;

Escribir Sin Saltar "Elija una contraseña de usuario nueva: ";

Leer pass;

encontrado <- Falso;

Para i <- 0 Hasta totalUsuarios - 1 Con Paso 1 Hacer

Si usuarios[i] == user Entonces

encontrado <- Verdadero;

FinSi

FinPara

Si encontrado Entonces

Escribir "";

Escribir "Ese usuario ya existe. Intente iniciar sesión.";

Sino

usuarios[totalUsuarios] <- user;

```
contras[totalUsuarios] <- pass;
```

```
totalUsuarios <- totalUsuarios + 1;
```

```
Escribir "";
```

```
Escribir "Cuenta creada con exito, bienvenido, ", user;
```

```
usuarioRetorno <- user;
```

```
opcion <- 3; // Sale del repetir
```

```
FinSi
```

```
Sino
```

```
Escribir "Opción inválida.";
```

```
FinSi
```

```
FinSi
```

```
Hasta Que opcion == 3
```

```
FinSubProceso
```

```
SubProceso TaqueriaPedidos(fechaActual)
```

```
Definir continuar Como Cadena;
```

```
Definir numeroMesa, opcion, cantidad Como Entero;
```

```
Definir subtotal, totalDia, precio, cargoExtra Como Real;
```

```
Definir resp Como Cadena;
```

```
Definir numPedidos Como Entero;
```

```
totalDia <- 0;
```

```
numPedidos <- 0;
```

```
continuar <- "si";
```

```
Mientras Minusculas(continuar) == "si" Hacer
```

```
  Escribir "";
```

```
  Escribir "=====";
```

```
  Escribir "  TAQUERÍA LAS BRASAS DE OAXACA  ";
```

```
  Escribir "=====";
```

```
  Escribir Sin Saltar "Ingrese el número de mesa: ";
```

```
  Leer numeroMesa;
```

```
  subtotal <- 0;
```

```
  opcion <- 0;
```

```
Mientras opcion <> 6 Hacer
```

```
  Escribir "";
```

```
  Escribir "--- MENÚ DE PLATILLOS ---";
```

```
  Escribir "1. Orden Tacos al Pastor (3 pzs) - $60";
```

```
  Escribir "2. Orden Quesadillas Oaxaqueñas (2 pzs) - $50";
```

```
  Escribir "3. Alambre Especial de la Casa - $110";
```

```
  Escribir "4. Agua Fresca (1 L) - $30";
```

```
  Escribir "5. Refresco (600 ml) - $25";
```

```
  Escribir "6. Finalizar pedido de la mesa";
```

```
  Escribir Sin Saltar "Seleccione una opción (1-6): ";
```

Leer opcion;

Si opcion ≥ 1 Y opcion ≤ 5 Entonces

Escribir Sin Saltar "Ingrese la cantidad solicitada: ";

Leer cantidad;

Si cantidad > 0 Entonces

cargoExtra $\leftarrow 0$;

precio $\leftarrow 0$;

Segun opcion Hacer

1:

precio $\leftarrow 60$;

Si cantidad $== 1$ Entonces

Escribir Sin Saltar "¿Desea 1 pieza individual? (s/n): ";

Leer resp;

Si Minusculas(resp) $== "s"$ Entonces

precio $\leftarrow 25$;

cargoExtra $\leftarrow 10$;

FinSi

FinSi

2:

precio $\leftarrow 50$;

Si cantidad == 1 Entonces

Escribir Sin Saltar "¿Desea 1 pieza individual? (s/n): ";

Leer resp;

Si Minusculas(resp) == "s" Entonces

precio <- 35;

cargoExtra <- 10;

FinSi

FinSi

3:

precio <- 110;

4:

precio <- 30;

5:

precio <- 25;

FinSegun

subtotal <- subtotal + (precio * cantidad);

Si cargoExtra > 0 Entonces

subtotal <- subtotal + cargoExtra;

Escribir "-> Se aplicó un cargo extra de \$10.00 MXN.";

FinSi

Escribir "-> Subtotal mesa: \$", subtotal;

Sino

Escribir "La cantidad debe ser mayor a 0.";

FinSi

Sino

Si opcion <> 6 Entonces

Escribir "Opción inválida.";

FinSi

FinSi

FinMientras

Si subtotal > 0 Entonces

totalDia <- totalDia + subtotal;

numPedidos <- numPedidos + 1;

Escribir "";

Escribir "RESUMEN MESA ", numeroMesa, " | Total: \$", subtotal;

Escribir "-> Venta registrada para la fecha: ", fechaActual;

Sino

Escribir "Pedido cancelado o sin ítems.";

FinSi

Escribir "";

Escribir Sin Saltar "¿Desea atender a otra mesa? (si/no): ";

Leer continuar;

FinMientras

Escribir "";

Escribir "---- CORTE DEL DÍA ----";

Escribir "Mesas atendidas: ", numPedidos, " | Ventas totales: \$", totalDia;

FinSubProceso